

LEVEL 20

Chapter 23: Performance Engineering

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 23: Performance Engineering

← [Chapter 22: Search Systems](#) · [Index](#) · Next → [Chapter 24: Resource Library](#)

23.1 The Bottleneck-Finding Habit

Nearly every "this system is slow, what do you do" interview moment rewards the same discipline: **don't guess, localize**. A request touches many layers (network, app CPU, database, cache, downstream services) — the skill being tested is whether you have a systematic way to figure out *which* layer is actually responsible before proposing a fix, rather than pattern-matching "add a cache" or "scale horizontally" reflexively (Chapter 14.4's most common mistake, applied specifically to this scenario).

The systematic approach to say out loud: "I'd want to look at where time is actually being spent before optimizing — using distributed tracing (Chapter 11.1) to see the breakdown across the request's full path, since guessing which layer is slow without that data risks optimizing the wrong thing entirely." This single sentence, said unprompted, is a strong signal — it shows your instinct is measurement-first, which is exactly the instinct your Prometheus/Grafana/OTel background should make natural to articulate.

23.2 CPU-Bound vs. I/O-Bound — The First Fork

	CPU-bound	I/O-bound
Symptom	High CPU utilization, request latency scales with computational work	Low CPU utilization but high latency — the process is <i>waiting</i> , not computing (on a DB query, a network call, a disk read)
Fix direction	Optimize the algorithm/code, or scale horizontally (more compute), or offload to more efficient hardware/language for hot paths	Fix the thing being waited on (Chapter 5's indexing/query optimization, Chapter 4's caching) or parallelize the waiting (concurrent requests instead of sequential)
Common interview trap	Adding more app server instances when the real bottleneck is a single, un-indexed database query every instance is waiting on — horizontal scaling doesn't help if everyone's blocked on the same downstream resource	—

Why this distinction matters as your first diagnostic fork: the fix for a CPU-bound problem (more/faster compute) actively *doesn't help* an I/O-bound problem, and vice versa — scaling out app servers when they're all blocked waiting on the same slow database query just means more requests queuing up behind the same bottleneck, not more throughput.

23.3 Database and Query Optimization

Most of this is a direct callback to Chapter 5 — here's the checklist form for a live interview:

1. **Is there a missing or wrong index?** (Chapter 5.1) — the single most common real-world fix, worth naming first.
 2. **Is the query fetching more than it needs?** (`SELECT *` instead of needed columns, missing `LIMIT`, no covering index forcing a bookmark lookup.)
 3. **Is there an N+1 query problem?** — a very common, very real bug pattern: fetching a list of 100 orders, then looping and issuing 100 separate queries for each order's items, instead of one join or one batched `WHERE order_id IN (...)` query. Worth naming explicitly — it's exactly the kind of thing that looks fine in a code review on a small dataset and becomes a severe bottleneck in production at scale.
 4. **Is the connection pool sized correctly?** (Section 23.5 below.)
 5. **Would denormalization or a read-optimized replica/materialized view help** for a specific expensive, frequent read pattern? (Chapter 5.1's explicit trade-off.)
 6. **Is this the point where caching (Chapter 4.5) should sit in front of the query entirely**, rather than continuing to optimize the query itself?
-

23.4 Caching for Performance — Beyond Chapter 4's Correctness Focus

Chapter 4.5 covered caching primarily through a correctness/consistency lens (stampede, penetration, avalanche). The performance-specific angle: **cache the expensive thing, not just the frequently-accessed thing** — these aren't always the same. A cheap query run frequently might not be worth caching (the cache-management overhead can exceed the query cost); an expensive aggregation run rarely might still be worth caching if any single instance of it is slow enough to matter to a user waiting on it. Size the caching decision to **cost × frequency**, not frequency alone.

Multi-layer caching: production systems commonly layer caches — a local, in-process cache (fastest, smallest, per-instance) in front of a distributed cache (Redis — larger, shared, one network hop) in front of the database. This tiered approach directly addresses

the hot-key problem from Chapter 4.5 (an in-process cache absorbs the very hottest keys without even a Redis round trip) while the distributed layer handles the broader working set.

23.5 Connection Pooling

The problem: establishing a new database (or any TCP-based) connection is relatively expensive (a TCP handshake, and for TLS-encrypted connections, a TLS handshake on top — Chapter 1.2) — doing this fresh for every single request is wasteful and adds real latency.

The fix: maintain a **pool** of already-open connections that requests borrow and return, rather than opening/closing per request. The sizing trade-off is genuinely two-sided and worth stating precisely: **too small** a pool means requests queue up waiting for an available connection even though the database itself has spare capacity (an artificial bottleneck you created); **too large** a pool can overwhelm the database with more concurrent work than it can efficiently handle, or exhaust the database's own max-connections limit across many app instances each holding their own pool (a genuinely common real-world outage cause — dozens of app instances × a generous pool size each, summing to far more connections than the database was provisioned for).

Interview question: "You doubled your app server instance count to handle more load, and now your database is falling over even though CPU on the database looks fine. Why?"

Ideal senior answer: "Almost certainly a connection pool exhaustion problem — if each app instance maintains its own connection pool of, say, 20 connections, doubling instances doubles total connections against the database, which may now exceed its configured max-connections limit or simply create more concurrent work than its actual capacity, even if raw CPU looks idle (the bottleneck could be lock contention or I/O wait, not CPU). I'd look at total connection count against the database's limit first, and consider a centralized connection pooler (like PgBouncer for Postgres) that multiplexes many app-level connections over a smaller number of actual database connections, rather than each instance maintaining its own large, uncoordinated pool."

23.6 Batching, Compression, and Async Processing

Batching: combining multiple small operations into one larger one to amortize fixed overhead — a database bulk-insert instead of N individual inserts, a Kafka producer batching several messages into one network send (Chapter 3.4's latency/throughput trade-off, applied directly: batching improves throughput at the cost of added latency for the batch to fill, so the batch size/wait-time is a real tuning knob, not a free win).

Compression: trading CPU time for reduced network/storage bytes — worth knowing when it's a clear win (large text payloads, logs, over slower/metered network links) versus when it's not (already-compressed content like JPEG/video, or extremely latency-sensitive small payloads where the compression/decompression CPU cost isn't worth the bytes saved).

Async processing: moving non-critical-path work off the synchronous request (Chapter 8.2's sync/async framework applied specifically as a *performance* lever, not just an architecture one) — the fastest possible synchronous response is one that does the absolute minimum required before responding, deferring everything else to a queue-backed background job.

23.7 Load Testing

Why it matters even though it's "not really an interview design topic": capacity estimation (Chapter 3) tells you what you *think* the system needs to handle; load testing is how you validate that the actual, built system genuinely handles it — worth mentioning briefly as part of a mature engineering process ("I'd validate this against a load test hitting the estimated peak RPS before considering it production-ready, not just trust the back-of-envelope math") since it signals operational maturity beyond the whiteboard design itself.

What good load testing checks for, beyond "does it survive": where the *first* bottleneck actually appears as load increases (validating or correcting your Chapter 14 Step 7 bottleneck prediction against reality), how the system behaves at and past its limit (graceful degradation via the patterns in Chapter 6.3, or a hard, ugly failure?), and whether autoscaling/alerting actually triggers correctly under realistic load patterns, not just a clean linear ramp.

23.8 The Interview Bottleneck-Identification Script

When asked "where would this design break first at 10x scale," work the layers in this order, out loud:

1. **Database writes** — usually the earliest hard ceiling in most designs (Chapter 5.5's scaling ladder).
2. **A specific hot key/partition** — even a well-distributed system can have one disproportionately popular entity (Chapter 4.5 and 5.4's hot-key/hot-partition discussions).
3. **A synchronous dependency chain** — any place a service waits on another service's response is a potential cascading-failure point (Chapter 8.6).
4. **A single-instance or under-provisioned stateful component** — anything that isn't trivially horizontally scalable (a single Redis instance not yet clustered, a connection pool sized for today's traffic).
5. **Network/bandwidth**, specifically for media-heavy systems (Chapter 3.3's video streaming example) — sometimes the binding constraint isn't compute or storage at all.

Walking this list in order, and picking the one or two that are actually plausible for *this specific* system rather than reciting all five generically, is the difference between a mechanical answer and a genuinely diagnostic one.

Chapter 23 Interview Drill

1. Explain why scaling app servers horizontally doesn't fix an I/O-bound bottleneck, with a concrete example.
 2. Walk through diagnosing and fixing an N+1 query problem.
 3. Explain connection pool sizing trade-offs in both directions (too small, too large).
 4. When would you choose not to compress a payload, even though compression is "free" bandwidth savings?
 5. Recite the 5-layer bottleneck-identification script from memory.
-

Next → [Chapter 24: Resource Library](#) — the best English and Hindi/Hinglish YouTube channels, blogs, books (with exact chapters to read), and courses, organized by learning stage.